

# MoE-Hub: Taming Software Complexity for Seamless MoE Overlap with Hardware-Accelerated Communication on Multi-GPU Systems

Zhuoshan Zhou<sup>1</sup>, Chen Zhang<sup>1\*</sup>, Shuyi Zhang<sup>1</sup>, Qijun Zhang<sup>1†</sup>, Haibo Wang<sup>2</sup>, Zhe Zhou<sup>2</sup>, Zhipeng Tu<sup>2</sup>, Guangyu Sun<sup>3</sup>, Yijia Diao<sup>1</sup>, Zhigang Ji<sup>1</sup>, Jingwen Leng<sup>1,4</sup>, Guanghui He<sup>1</sup>, Minyi Guo<sup>1</sup>  
Shanghai Jiao Tong University<sup>1</sup>, Huawei Technologies Co. Ltd.<sup>2</sup>,  
Peking University<sup>3</sup>, Shanghai Qi Zhi Institute<sup>4</sup>

{zs.zhou, chenzhang.sjtu, sy.zhang, diao\_yijia, zhigangji, guanghui.he}@sjtu.edu.cn, qijunzhang2000@gmail.com,  
{wanghaibo33, zhouzhe22, tuzhipeng3}@huawei.com, gsun@pku.edu.cn, {leng-jw, guo-my}@cs.sjtu.edu.cn

**Abstract**—The Mixture-of-Experts (MoE) architecture is crucial for scaling large language models, but its scalability is severely limited by inter-GPU communication bottlenecks in multi-GPU systems. Although overlapping communication with computation is a widely recognized optimization, its effective deployment still remains challenging, both in terms of performance and programmability. In this work, we identify the root cause as a fundamental abstraction mismatch between MoE’s dynamic, irregular token-to-expert mapping and the static, address-centric communication model of modern GPUs, which necessitates a complex software mediation phase to resolve addresses before data transfers, limiting performance and software flexibility. To resolve this, we propose MoE-Hub, a hardware-software co-design that introduces a destination-agnostic communication paradigm. MoE-Hub decouples data transmission from address management, allowing producers to send data immediately after routing using only a logical destination, while address allocation and data-flow orchestration are handled transparently by lightweight hardware in the GPU hub. By hardware-accelerating the entire communication control plane, MoE-Hub enables seamless and transparent overlap. Our evaluation shows that MoE-Hub achieves 1.40×–3.08× per-layer and 1.21×–1.98× end-to-end speedup over state-of-the-art systems.

**Index Terms**—Multi-GPU Architecture, Mixture-of-Experts (MoE), Fine-grained Overlap

## I. INTRODUCTION

The scaling law for large language models (LLMs) has demonstrated that model capability correlates strongly with parameter count and training data size [27]. However, the computational burden of dense Transformer models grows prohibitively with scale, outpacing the capabilities of modern accelerators. The Mixture-of-Experts (MoE) architecture has emerged as a transformative solution to this challenge [12], [13], [31], [53], [56]. By replacing the dense feed-forward network (FFN) in a Transformer layer with a sparse layer comprising multiple experts and dynamically routing each token to only a small subset (e.g., Top-K), MoE models achieve the capacity of trillion-parameter networks while incurring only a

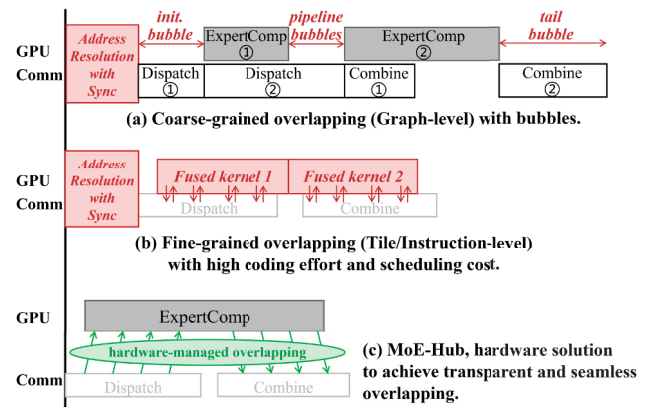


Fig. 1: Comparison of computation-communication overlap strategies in MoE systems.

fraction of the computational cost [12], [13], [22], [55]. This approach has been validated by state-of-the-art models such as Mixtral-8x7B [22], DeepSeek [9]–[11], GPT [49], [50], Llama 4 [36], DBRX [54] and others [1], [4], [45], [60], firmly establishing MoE as a crucial component in modern LLMs.

To accommodate the massive parameter footprint of these models, expert parallelism is required with experts distributed across the memory of multiple GPUs. While this alleviates memory pressure, it introduces a significant performance challenge owing to the extensive inter-GPU communication required. The forward pass of popular MoE models can spend an average of 47% of its execution time on device-to-device data exchange during the All-to-All dispatch and combine phases [67], shifting the bottleneck from computation to communication.

A canonical strategy to mitigate communication overhead is to overlap it with computation. Prior works have explored this along two primary trajectories, as summarized in Fig. 1. Coarse-grained methods pipeline tensor slices at the computation-graph level [16], [19], [57], [58], [68], but they suffer from pipeline bubbles due to unpredictable com-

\* Chen Zhang is the corresponding author.

† Qijun Zhang participated in this project during his internship at Shanghai Jiao Tong University.

munication and compute loads caused by MoE’s dynamic routing, which changes token-to-expert mappings with each input. On the other hand, fine-grained approaches seek to fuse All-to-All communication with expert computation within dedicated kernels, scheduling at the tile or instruction level [2], [61], [67]. While these methods improve overlap, they incur significant software scheduling overhead, requiring complex, hardware-specific orchestration of synchronization and memory accesses, limiting performance and portability. Both approaches struggle to deploy MoE models efficiently.

Our analysis reveals that these overheads are not mere implementation shortcomings but symptoms of a deeper, fundamental abstraction mismatch. The MoE algorithm specifies a *dynamic, irregular mapping* from tokens to experts, whereas GPU interconnects rely on a *static, address-centric* communication paradigm requiring explicit underlying memory addresses as destinations. This mismatch forces a costly software-mediated coordination that inter-device synchronizations are required before communication can even begin, to collectively determine the address mappings for all tokens on consumer GPUs. The prerequisite synchronization, along with the ensuing software complexity for managing fine-grained data, severely limits the efficacy and transparency of overlap, creating a significant gap between the performance of state-of-the-art systems and the theoretical performance limit.

In this paper, we propose MoE-Hub, a hardware-software co-design that introduces a new communication abstraction to resolve this mismatch. The core idea is to shift from an *address-centric* to a *destination-agnostic* communication paradigm, decoupling data movement from address allocation. This allows producers to initiate data transfers immediately upon obtaining a token’s routing result, without knowing its final memory address, while address placement is handled transparently by hardware on the consumer side. Moreover, we accelerate the entire data flow through hardware enhancements to achieve high-performance overlap, including congestion- and consumer-aware packet management for producers, and a lightweight producer-consumer signaling mechanism that triggers consumer computation when data becomes available.

In this work, we make the following contributions:

- We identify and formalize the semantic mismatch between the producer-consumer model of MoE algorithms and the communication abstraction of modern GPUs as the root cause of inefficiency in expert parallelism.
- We propose MoE-Hub, a holistic design that integrates three key techniques: (i) ISA and microarchitectural support for destination-agnostic communication paradigm; (ii) a runtime packet manager to optimize bandwidth efficiency and transmission order; and (iii) a data availability manager to provide a hardware signaling mechanism between producers and consumers.
- We implement MoE-Hub on a cycle-accurate multi-GPU simulator and evaluate it using three representative MoE models. MoE-Hub achieves a  $1.40\times$ – $3.08\times$  speedup per MoE layer and  $1.21\times$ – $1.98\times$  end-to-end speedup over state-of-the-art software methods.

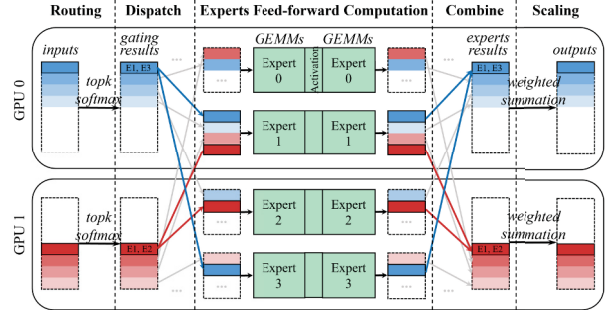


Fig. 2: Exemplary Execution of a 4-expert MoE layer distributed across 2 GPUs (Top-2 routing).

The rest of this paper is organized as follows: Section II provides background and a quantitative analysis of existing work’s limitations. Section III presents our key insights and design philosophy. Section IV details the MoE-Hub architecture. Sections V and VI describe our methodology and present a comprehensive evaluation. Finally, we discuss extensions of the proposed mechanism in Section VII, related work in Section VIII, and conclude in Section IX.

## II. BACKGROUND AND ANALYSIS

### A. MoE Structure and Expert Parallelism

MoE model replaces the dense Transformer feed-forward network (FFN) with a set of experts and a router that sparsely activates only a small subset of experts per token, enabling substantial capacity scaling with bounded per-token compute [12], [13], [31], [53], [56]. The architecture is widely recognized by the research community [2], [16], [18]–[20], [23], [26], [32], [34], [52], [57], [61], [67], industry [4], [9], [36], [49], [50], [69], and hardware vendors [3], [41], [42], [46] as a promising approach for scaling LLMs. MoE scaling relies on expert parallelism (EP), which shards experts across GPUs or nodes to manage growing parameters. This introduces all-to-all collectives whose efficiency becomes the bottleneck limiting end-to-end throughput at scale.

Fig. 2 illustrates a typical execution of a distributed MoE layer, consisting of five essential steps in the algorithm [61]: ① Routing. The output token sequence from the attention module is partitioned and distributed across GPUs along the sequence dimension. These tokens first pass through a routing network to determine expert assignments, typically involving a gating network to compute token–expert scores, followed by normalization and selection of the top- $k$  experts [13], [55], [56]. ② All-to-All Dispatch. Each token is dispatched to the selected experts, with the dispatch phase in EP forming a non-uniform all-to-all communication across GPUs. ③ Experts Computation. Each expert performs computation on its tokens, including two GEMM operations with an activation function in between. ④ All-to-All Combine. The reverse of dispatch that transfers tokens back to their original positions, forming another all-to-all communication in EP. ⑤ Scaling. Each token aggregates outputs from experts through a weighted sum, with weights from the expert scores computed during routing.

TABLE I: Coding Effort: SOTA vs. MoE-Hub.

| MoE Systems           | Address Resolution | Scheduling (host / device) | Communication (host / device) |
|-----------------------|--------------------|----------------------------|-------------------------------|
| CCFuser [61]          | CPU                | 603 / 211                  | 24 / 34                       |
| Comet [67]            | CPU                | 6347 / 5589                | 1197 / 1305                   |
| DeepEP [69]           | CPU                | 1604 / 3326                | 498 / 1899                    |
| FlashDMoE [2]         | GPU                | 720 / 1706                 | 513 / 1137                    |
| Primus-Turbo [3]      | GPU                | 1341 / 853                 | 386 / 2260                    |
| <b>MoE-Hub (ours)</b> | <b>GPU</b>         | <b>0</b>                   | <b>&lt;10 (store insts.)</b>  |

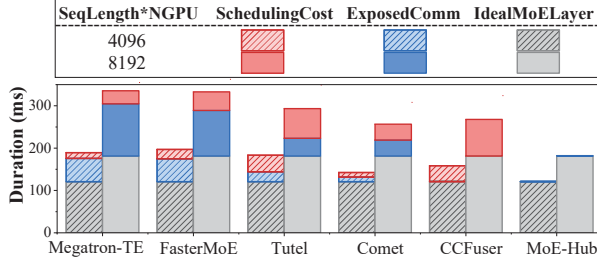


Fig. 3: MoE performance gap to ideal (Mixtral-8x7B on  $8 \times \text{H800}$ ), dominated by software scheduling and exposed communication.

### B. The Challenge of Overlapping in Expert Parallelism

Costly dispatch and combine communications across devices significantly degrade MoE model performance. A common strategy to mitigate communication latency is to overlap it with computation. Existing approaches broadly fall into two categories, each with inherent limitations when applied to the dynamic and irregular patterns of MoE models.

1) *Coarse-Grained Overlap*: Early efforts achieve overlap at the computation-graph level by pipelining tensor slices [16], [19], [57], [68], where input tensors are partitioned into smaller slices and processed in a pipelined manner, enabling the computation and communication of different slices to proceed concurrently. However, MoE’s dynamic routing mechanism causes varying expert selections for each input, leading to fluctuating communication volumes and changing expert computation workloads. Unlike static scheduling in other parallelisms, such as tensor or pipeline parallelism, where coarse-grained computation and communication can be effectively overlapped, this inherent variability often introduces pipeline bubbles in tensor-slicing pipelines. With kernel launch overhead limiting the number of slices, the bubble problem can become significant and cause severe resource under-utilization.

2) *Fine-Grained Overlap*: Recent studies demonstrate a trend toward finer-grained overlap, where scheduling occurs at the tile or instruction level. These works can be further divided into two sub-categories by implementation strategy.

#### a) Kernel Fusion with Software-Managed Pipelines:

This approach coordinates computation and communication within fused kernels through software-managed pipelines [2], [61]. Programmers typically rely on a global address space abstraction and implement fine-grained inter-device communication using low-level memory access semantics, such as one-sided communication APIs provided by libraries (e.g.

NVSHMEM [47]), or issuing direct peer-to-peer (P2P) memory requests between supported devices [40].

b) *Tile-Level Scheduling with Dedicated Resources*: Another line of work seeks to avoid repeatedly re-implementing low-level communication details by scheduling computation-communication overlap at the tile level [67]. A common approach is to dedicate a subset of streaming multiprocessors (SMs) exclusively to communication, while the remaining SMs perform computation, with synchronization signals manually implemented between the two groups.

While these fine-grained methods represent a significant advance over coarse-grained pipelining, they impose substantial software complexity and non-negligible performance overheads, which we analyze in depth in the next section.

### C. Analysis of Existing Solutions and Their Limitations

Before analyzing existing solutions, we define an *ideal MoE layer* as performing only the essential operators on local tokens (routing, expert computation, and scaling), with dispatch and combine communication fully hidden and no additional steps. This allows us to separate the core algorithmic requirements from the implementation-specific overheads.

1) *Heavy Software Engineering Burden*: To quantify the software engineering burden, Table I analyzes the lines of code (LoC) dedicated to scheduling and communication in state-of-the-art systems, excluding the core MoE operations of an *ideal MoE layer*. The results show that scheduling overlap pipelines, involving SM coordination, memory barriers, and kernel launch ordering, require substantial code, with even leaner systems needing hundreds of lines. Furthermore, fine-grained communication is inherently complex, as demonstrated by DeepEP [69], a dedicated All-to-All communication library, which requires extensive device code to ensure efficient inter-GPU data movement. In stark contrast, our proposed MoE-Hub requires almost no scheduling code and only minimal communication instructions, demonstrating the potential of a hardware-native abstraction to drastically reduce software complexity and improve portability.

2) *Intrinsic Performance Overhead*: Beyond development cost, the runtime overhead of software scheduling imposes a performance ceiling. Fig. 3 decomposes the end-to-end performance gap between state-of-the-art systems and ideal implementation with *ideal MoE layers*, highlighting two key observations. First, a large portion (in red) is attributed to algorithm-unrelated software scheduling overhead, including extra data manipulations, synchronization between devices, and multiple kernel launch latencies. Second, a substantial fraction (in blue) remains as exposed communication that cannot be fully overlapped due to the inflexibility of software pipelines in handling dynamic, fine-grained data flows. Collectively, these overheads account for over 24% of the total MoE layer time, even in highly optimized implementations. It is not merely an implementation artifact but a direct consequence of fundamental misalignment between MoE algorithms and modern GPU communication model. As system scale and

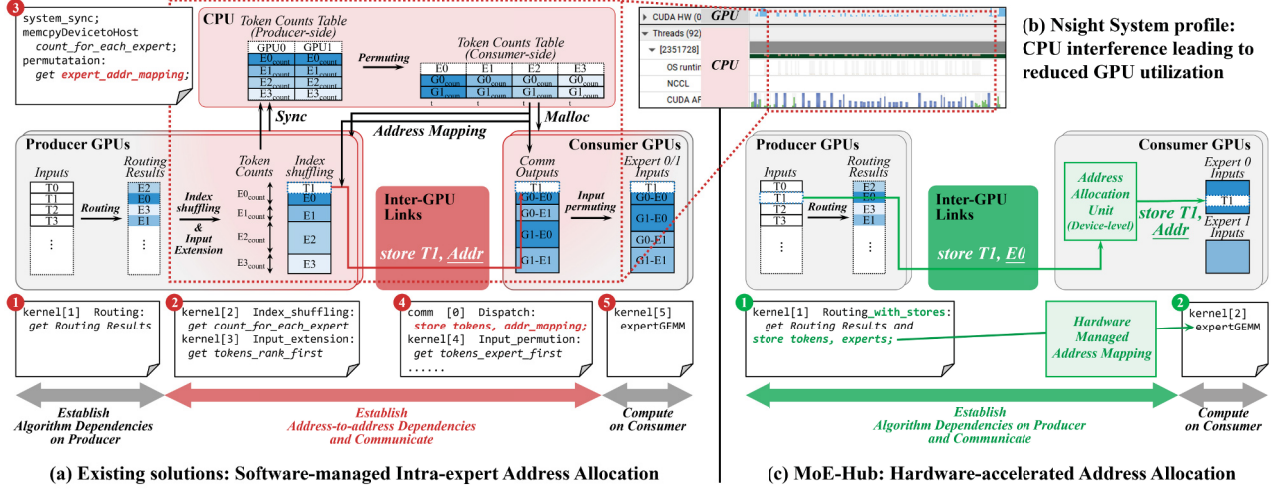


Fig. 4: From software-mediated address resolution (a, b) to hardware method (c), MoE-Hub eliminates heavy software overheads.

model heterogeneity grow, this overhead will amplify, limiting the efficiency and scalability of MoE models.

### III. INSIGHTS AND DESIGN PHILOSOPHY

Our analysis in Sec II reveals that software-based overlap techniques incur heavy overhead. We identify that this overhead stems from two fundamental sources: a semantic mismatch between the algorithmic and hardware communication models, and the inherent inefficiency of software in managing fine-grained, dynamic data flows. In this section, we distill these problems into core insights and present the overarching design philosophy of MoE-Hub, which guides our holistic hardware-software co-design.

#### A. Insight I: Mismatch in Producer-Consumer Abstraction

The root cause of software complexity lies in a fundamental semantic mismatch between the *dynamic, irregular* token-to-expert dependencies in MoE algorithms and the *static, address-to-address* communication model enforced by GPU hardware. The MoE routing algorithm only determines which expert on which GPU should process a token; it does not specify the token's exact position within the expert's input tensor. However, inter-GPU communication, which operates on a direct memory access model with load/store semantics, requires the producer to know the precise destination memory address before any data transfer can occur.

The mismatch forces a costly software mediation phase to compute memory addresses before initiating an All-to-All dispatch. Fig. 4(a) illustrates a CPU-coordinated example. Although the routing result is already available, e.g., token T1 should be sent to expert E0 on GPU2, the token still cannot be transmitted immediately, because its exact destination address in remote GPU remains unknown. Computing this address forces a complex procedure: all tokens synchronize, shuffle, and await CPU-driven memory layout before deriving per-token offsets. With the root cause being the need to resolve

addresses at runtime, this tortuous data flow creates software complexity and overhead, as profiled in Fig. 4(b).

An alternative is to adopt a fully GPU-resident mediation stack, e.g., back-to-back index all-gather and data all-to-all collectives like Primus-Turbo [3], or fused kernels like FlashD-MoE [2]. However, these approaches still require dynamic address coordination and merely shift the aforementioned software complexity to CUDA code. For example, the second all-to-all phase in back-to-back designs cannot utilize a standard library operator and must be hand-crafted for irregular, runtime-dependent message sizes and issue timing, complicating communication overlap. As shown in Table I, software mediation increases development effort, reduces portability, and may still incur non-trivial overhead. These overheads persist because routing results vary per input, dynamically changing each token's expert assignment and each expert's incoming token set and load, preventing any precomputed static address mapping from being directly used.

In summary, under the existing address-to-address communication model, the dynamics of MoE routing necessitate a convoluted runtime address resolution before data movement can begin. Consequently, even with routing results in hand, data transmission is blocked, preventing immediate overlap. All existing software solutions attempting to manage this mismatch incur significant orchestration overhead, as analyzed in Sec. II-C.

**Insight-1:** The dependency for communication is the expert ID, NOT the memory address. By decoupling these two, we can eliminate the complex software-mediated address resolution phase.

#### B. Insight II: Inefficiency in Fine-Grained Data Management

Software scheduling is further hampered by its inability to efficiently manage the fragmented and out-of-order data flows inherent to MoE, impacting both producers and consumers.

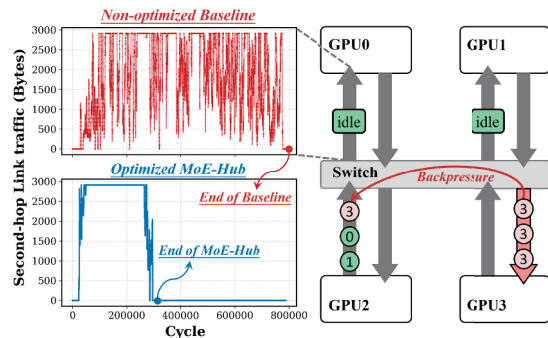


Fig. 5: MoE’s dynamic, irregular routing often causes traffic bursts and congestion, degrading bandwidth utilization.

**Producer-Side Transmission Inefficiency:** Eliminating software address coordination enables earlier data communication, but this advantage is challenged by the intrinsic dynamics in MoE routing stages. The stochastic nature of MoE routing causes each producer to generate a flood of fine-grained, out-of-order remote memory requests that dispatch tokens to arbitrary GPUs. This results in highly irregular traffic patterns that, without proper hardware orchestration, degrade performance in two critical ways. First, on the producer side, traffic bursts and routing randomness lead to congestion on specific consumer GPU $\leftrightarrow$ Switch links. This congestion creates backpressure to the producer GPU, which in turn impedes data transmission to other consumers, ultimately reducing overall bandwidth utilization, as illustrated in Fig. 5. Second, on the consumer side, the arrival of incomplete or misordered tokens, misaligned with the consumer’s computation granularity, delays the initiation of expert kernels. Managing such packet-level interleaving in software is infeasible in real time without introducing costly global synchronization.

**Consumer-Side Polling Overhead:** On the consumer side, the fine-grained arrival of tokens forces expert kernels to continuously poll for data availability. As shown in Fig. 6, this polling can consume a significant portion of consumer execution, occupying memory bandwidth and compute cycles, as numerous warps remain active solely to check semaphores. This “busy-waiting” problem intensifies as communication granularity shrinks, directly stealing resources from actual computation.

**Insight-2:** Managing fine-grained, dynamic data flows requires hardware-level support for real-time packet management and signaling mechanism to achieve seamless overlap.

### C. Design Philosophy

Guided by these insights, our design philosophy is to offload the root causes of overhead into a dedicated hardware substrate, thereby introducing a new communication abstraction that is native to the MoE workload.

The cornerstone of this philosophy is a shift from an *address-centric* to a *destination-agnostic* communication

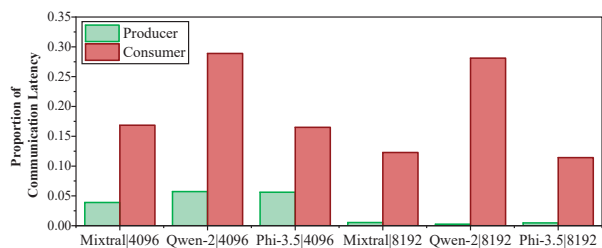


Fig. 6: Proportion of communication latency on both producer and consumer sides of dispatch phase.

paradigm. As illustrated in Fig. 4(c), we *decouple data transfer from address allocation*. This allows a producer to initiate a transfer immediately upon obtaining a token’s routing result Expert ID, along with the GPU ID hosting that expert, without knowing its final memory address. The consumer’s hardware then dynamically allocates a local address upon the data’s arrival. This decoupling alone eliminates the mediation phase previously required for address resolution.

However, to make this core abstraction efficient and transparent, the hardware substrate must be extended with three key capabilities, which form the pillars of our co-design stack:

- A logical addressing and on-demand allocation mechanism to manage the dynamic mapping from routing results to memory addresses on the consumer side.
- A runtime packet management unit on the producer side to schedule the fine-grained, irregular traffic.
- A hardware signaling mechanism on the consumer side to replace software polling by automatically triggering computation once its required data arrives.

By co-designing the instruction set, the microarchitecture, and the runtime system around these pillars, MoE-Hub enables transparent, fine-grained overlap without requiring low-level software orchestration, directly addressing the aforementioned semantic mismatch and management inefficiencies.

### D. Architectural Pillars

Translating our design philosophy into a practical architecture relies on three co-designed pillars that collectively decouple communication from address allocation and manage fine-grained data flows transparently. These pillars form the core of our hardware innovations.

- **Destination-Agnostic Communication ISA:** This pillar breaks conventional ISAs’ address-centric limitation by introducing a new store instruction that encodes a logical destination (e.g., Expert ID), coupled with a consumer-side *Address Allocation Unit (AAU)* for hardware-based address assignment upon data arrival. This co-design eliminates the costly mediation phase, enabling smooth and low-overhead producer-consumer data transfers.
- **Hardware-Accelerated Packet Management:** This pillar tackles the inefficiency of fine-grained, irregular token emissions through a hardware-based *Runtime Packet Manager (RPM)*. This unit reshapes the outgoing traffic

by partitioning packets by destination, coalescing them into interconnect-friendly bursts, and scheduling transmissions to balance load while prioritizing critical data, transforming irregular streams into efficient flows.

- **Fine-Grained Hardware Signaling:** This pillar replaces software polling with a hardware-based *Data Availability Manager (DAM)*. The DAM tracks fine-grained data dependencies and uses write acknowledgments to dispatch consumer thread blocks immediately upon data arrival, eradicating polling overhead and freeing valuable compute resources.

#### E. The Hub: An Architectural Locus

The three pillars described above collectively point to a common architectural requirement for a unified hardware locus situated at the intersection of computation, memory, and communication. The GPU hub serves as this foundational nexus. As a native component of the GPU architecture, the hub originally interfaces directly between the on-chip crossbar and interconnects (e.g., NVLink), and can connect to compute cores (SMs/L1) and the memory system (L2/memory controllers) through the crossbar. Fig. 7 illustrates its privileged position in multi-GPU architecture. The central vantage point allows the hub to observe all local and remote memory traffic, making it the ideal site for integrating the Address Allocation Unit, Runtime Packet Manager, and Data Availability Manager into a cohesive component. By extending the hub with our modules, we establish a unified substrate that seamlessly bridges computation and communication, realizing all architectural pillars efficiently.

### IV. MOE-HUB DESIGN

MoE-Hub embodies the design philosophy outlined in Section III by implementing a unified hardware substrate within the GPU hub. The architecture realizes the three co-designed pillars introduced earlier, including a destination-agnostic communication paradigm, hardware-accelerated runtime packet management, and fine-grained hardware signaling. Fig. 7 illustrates the overall architecture of MoE-Hub. The major newly introduced hardware modules are integrated into the GPU hub, including AAU, RPM, and DAM.

#### A. A Destination-Agnostic Communication Paradigm

The cornerstone of MoE-Hub is a new communication abstraction that directly addresses the semantic mismatch identified in Section III-A. Current inter-GPU communication, built upon Unified Virtual Address (UVA) [40], is fundamentally address-centric. The producer initiates remote writes using the same store instructions as for local memory accesses (Fig. 8(a)). After address lookup in the local TLB, remote store requests are routed not to the L2 cache or local DRAM but to the transmission unit hub for delivery to other devices. While unifying memory spaces, this mechanism requires the producer to know the exact requested address in the consumer's memory to issue the instruction. In MoE models, this necessitates

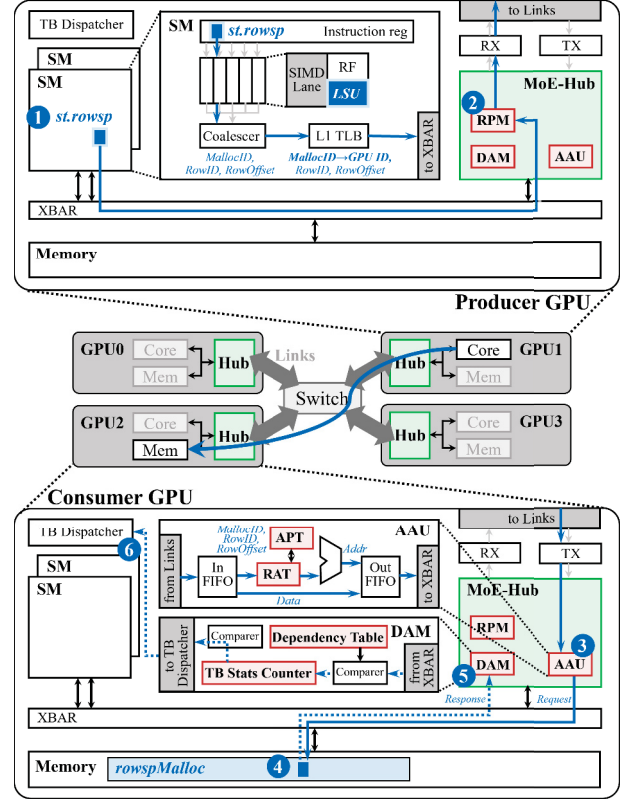


Fig. 7: MoE-Hub overview: hub-side extensions and datapath integration.

complex, software-mediated phases prior to communication to establish these address mappings across GPUs.

*Our key insight is that the true dependency for communication is the logical destination (Expert ID), not the underlying memory address.* We therefore introduce a destination-agnostic paradigm that decouples data transfer from address allocation, allowing communication to begin as soon as a token's routing result is available.

1) **ISA Extensions:** As shown in Fig. 8b, we introduce a new row-sparse store instruction, *st.rowsp*, where a *row* corresponds to one token in the activation tensor. Similar to a conventional store, *st.rowsp* employs one register for the destination, but that register contains three fields to specify a logical destination:

- **RowID:** A globally unique index assigned by programmer to identify a token row across GPUs and encode a desired transmission ordering among outstanding rows.
- **RowOffset:** As the row size is typically larger than the size of a memory request, a row is transmitted as multiple packets, with each up to 128 B over NVLink [7], [37], [38]. RowOffset specifies the offset within the row.
- **MallocID:** An identifier for a memory region allocated for dynamic address assignment. It encodes both the target GPU identifier and a region-specific identifier.

Apart from the logical destination, the new instruction

follows the same semantics as a conventional GPU store instruction. To further manage traffic priority, the ISA is extended with a `.nop` suffix (e.g., `st.rowsp.nop`) for non-critical-path data transfers. The transmission order among all priority requests is controlled by the programmer via the RowID, to avoid excessive tail latency for partial data within a token row.

**Datapath:** `st.rowsp` allows a producer to initiate a transfer as soon as a token's routing result (i.e. Expert ID) is known. Fig. 7 illustrates the end-to-end datapath of the proposed destination-agnostic instruction, which shares the same datapath with existing memory requests. Issued by producer SMs ①, they are processed by the SIMD lane load/store units, coalesced into transactions, and get the target peer-GPU identifier resolved from the logical destination in TLB. Transactions generated by `st.rowsp` are then forwarded to MoE-Hub, where the Runtime Packet Manager (RPM) ② processes them before interconnect transmission. On the consumer side, incoming destination-agnostic requests are handled by the Address Allocation Unit (AAU), which allocates store addresses on arrival within the pre-allocated region indexed by `MallocID` ③. In contrast, conventional remote stores bypass the AAU. The request is then forwarded to the consumer memory system ④. Similar to conventional remote stores, it first undergoes address translation via the IOMMU at the interface, then passes through the crossbar to reach the target memory and complete the write operation. Finally, write acknowledgments are captured by the Data Availability Manager (DAM) ⑤, which uses them to guide the TB dispatcher ⑥ for consumer thread-block scheduling.

2) **Runtime API:** A new runtime API, `rowspMalloc` (Fig. 8c), allocates a region of device memory for on-demand consumer-side address assignment and returns a `MallocID` to be used by `st.rowsp` instruction. This API performs two operations: (i) It registers the memory region's metadata, including the base address (`BaseAddr`) and row size (`RowSize`), directly in the target GPU's hub via memory-mapped I/O (MMIO), reusing the standard GPU driver programming interface for controlling device engines and state registers (e.g., engine setups and interrupt controls) to configure the AAU. (ii) It generates a `MallocID` that both identifies the target GPU and points out allocated memory regions. At runtime, `st.rowsp` uses `MallocID` to indicate routing destination in the producer and perform address allocation in the consumer. The resolution of `MallocID` to GPU identifier occurs in the L1 TLB of producer SMs. Unlike conventional remote stores that depend on TLB lookups to derive the peer GPU from a virtual address, the new instruction is routed via lightweight logic gated by an instruction flag, avoiding interfering with the standard address translation process. The address allocation within the pointed out region is performed in consumer AAU, as shown in Fig. 7.

The API `rowspMalloc` mainly serves as an interface to the GPU memory-management system. It exposes a region-based provisioning abstraction that allocates a device-memory region for on-arrival placement. Since MoE-Hub's

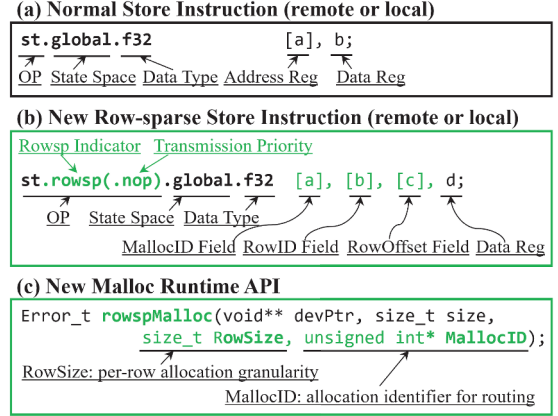


Fig. 8: ISA and runtime API extensions.

focus is on communication-model optimization, it employs a static memory management model, pre-allocating conservative expert-input regions to prevent token overflow. Behind the `rowspMalloc` API, a more advanced memory management scheme (e.g., paging-style KV-cache management or emerging GPU-side proposals [8], [15], [30], [33]) can be integrated to improve memory utilization, provided that two conditions are met: (i) `st.rowsp` requests are routed correctly, and (ii) on-arrival address allocation is computed correctly. We leave such extensions for future work.

3) **Architecture Support:** The logical abstraction of `st.rowsp` is translated into concrete addresses in hardware by the Address Allocation Unit (AAU) on the consumer side. The AAU is responsible for the on-demand, contention-free translation of logical (`MallocID`, `RowID`) tuples into real addresses, which comprises two key structures:

- Row Allocation Table (RAT): A tag-RAM that caches the mapping from (`MallocID`, `RowID`) to a locally allocated `LocalRowID`.
- Allocation Pointer Table (APT): A small CAM that tracks the next available `LocalRowID` for each `MallocID` using a `RowPointer`.

The two components jointly perform address allocation on the consumer GPU. Upon packet arrival at the destination hub, the AAU looks up the RAT for an existing `LocalRowID` corresponding to the same (`MallocID`, `RowID`). If a valid entry is found, the packet reuses the cached `LocalRowID`. On a RAT miss, AAU checks whether the entry was previously allocated. For entries that have never been allocated, a new `LocalRowID` is assigned using the `RowPointer` in the APT, which is then atomically incremented to ensure contiguous, conflict-free allocation. For evicted entries, the mapping is restored from memory. Final memory address is computed as `BaseAddr + LocalRowID * RowSize + RowOffset`. This mechanism ensures that tokens from all producers are densely packed in the consumer's memory based on their arrival order, without synchronization between devices. The `st.rowsp` instruction also supports local token

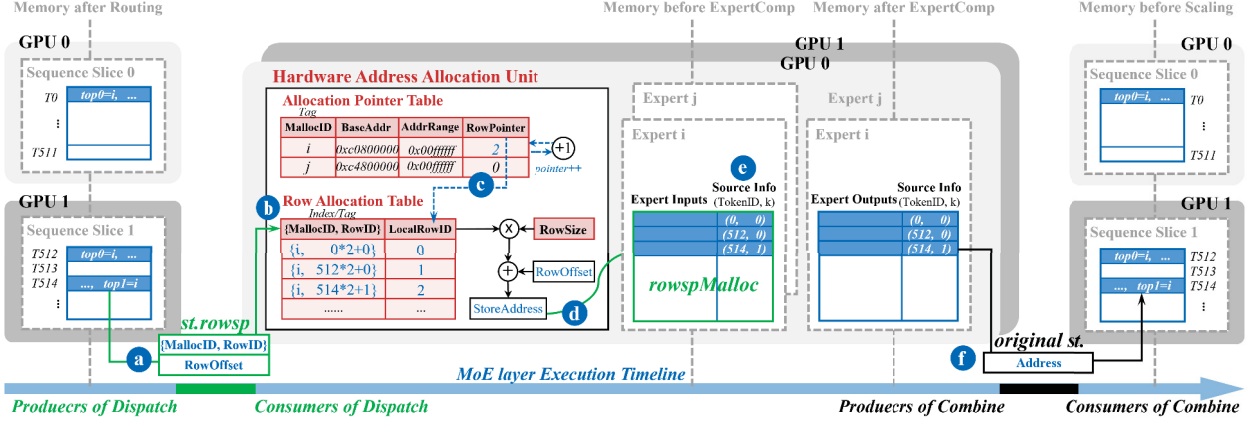


Fig. 9: Illustration of MoE layer dataflow under the destination-agnostic communication paradigm.

storage within local experts, maintaining a unified address mapping strategy for both remote and local tokens.

**Eviction Mechanism:** The eviction policy in MoE-Hub is triggered when a new entry needs to be allocated but the RAT table is full. Each RAT entry only needs to be retained briefly, until all packets for its corresponding row have arrived. Once a row is fully received, the mapping between (MallocID, RowID) and LocalRowID becomes unnecessary and can be safely evicted. Because smaller RowIDs are prioritized by the RPM on the producer side (detailed in IV-B), packets for the same row arrive at the receiver in quick succession, prompting the use of a first-in-first-out strategy to facilitate evicting entries that are no longer needed. Evicted mappings are spilled to device memory. If a late packet arrives after eviction, the mapping is recovered on demand, ensuring that all packets of the row resolve to the same LocalRowID. The RAT/APT is flushed only when the rowSpMalloc region is either freed or reinitialized. These operations are confined to hub metadata and do not require any changes to global coherence or memory consistency mechanisms.

4) *All-to-All Communication Workflow:* Figure 9 illustrates the end-to-end MoE layer workflow across two GPUs, enabled by the hardware and software support of MoE-Hub.

a) *All-to-All Dispatch:* The producer (routing kernel) initiates `st.rowsp` write requests to the target expert's GPU based on the logical routing computation results **a**. Upon reaching the target device's hub, the logical address of the request is translated by the AAU **b**. If no corresponding mapping exists, a new entry is allocated using the RowPointer stored in APT **c**, with the pointer then self-incremented. After obtaining LocalRowID, storage address is computed **d**. The `st.rowsp` request is queued for transfer through the on-chip XBAR to the L2 cache and memory for the actual write.

b) *All-to-All Combine:* This process involves communication in the opposite direction of dispatch. During dispatch, additional source information for the token is provided to ensure the token can be accurately routed back to its original location. Implemented as an extra column in the expert input activation tensor **e**, the source information undergoes

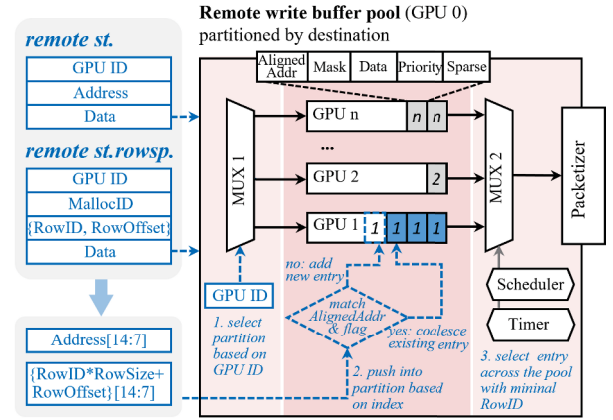


Fig. 10: Runtime Packet Manager microarchitecture.

logical address translation along with the token data, yet it is transferred using the `st.rowsp.nop` instruction, as it lies off the critical path. At the end of the expert's final GEMM, the expert can read this source metadata to initiate combine via conventional stores **f**.

#### B. Producer: Runtime Packet Manager

While the new communication paradigm enables immediate data transfer, the fine-grained, irregular communication pattern inherent in MoE workloads poses a fundamental challenge to interconnect efficiency. To this end, we introduce the Runtime Packet Manager (RPM) at the hub's egress point, which is specifically designed to mitigate the bandwidth underutilization identified in Section III-B. The RPM transforms bursts of small, out-of-order packets into a structured stream.

1) *Microarchitecture:* The manager's front-end is a Remote Write Buffer Pool, structured as a set of fully-associative SRAM partitions, each dedicated to an active peer GPU in the link fabric. The peer set is fixed at job initialization and bounded by the physical topology (e.g., 8–16 peers in current systems [44]). This design is critical for isolating and managing traffic on a per-link basis. Fig. 10 illustrates the

microarchitecture design and workflow for instructions. When a remote `st.` or `st.rowsp` request arrives, it is first routed to its corresponding destination partition.

- For remote `st.` requests, the request is indexed by its cache-line-aligned address. The buffer checks for an existing entry with a matching address.
- For remote `st.rowsp` requests, the request is indexed by its  $(\text{RowID}, \text{RowOffset})$  tuple, treating the logical row as its own "address space" for packet merging.

If a matching entry is found and the priority flags (e.g., standard vs. `.nop`) concur, the new request is merged into the existing entry by updating its validity mask. This minimizes protocol overhead by favoring the generation of interconnect-friendly packets (e.g., 128-byte cache lines).

2) **Packet Scheduling:** The buffer pool's back-end is a Packet Scheduler responsible for arbitrating which merged requests are transmitted onto the interconnect and when. Its scheduling policy is multi-faceted, being both congestion-aware and consumer-aware.

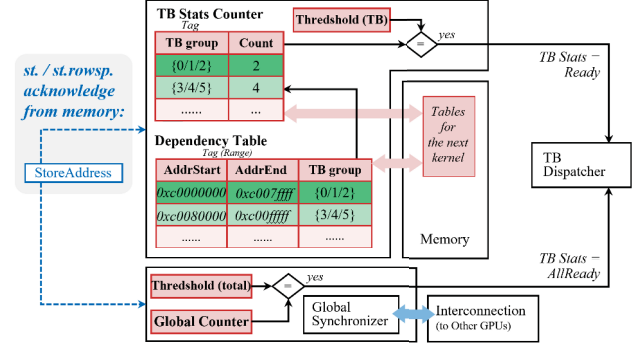
a) **Congestion-Aware Emission Strategy:** The scheduler uses a round-robin polling mechanism across buffer partitions for each destination GPU, smoothing traffic to prevent routing-induced sudden bursts toward a single GPU which would increase queueing latency in the interconnect and slow down overall transmissions, as shown in Fig. 5, thereby maintaining balanced utilization across links. It also optimizes transmission efficiency by prioritizing entries with fully set validity masks, which indicate maximum granularity merge. A timer-based bypass mechanism is incorporated to prevent poorly merged entries from stalling indefinitely in the buffer, maintaining system-wide robustness and ensuring forward progress.

b) **Consumer-Aware Priority Policy:** Within each destination buffer partition, the scheduler considers two levels of priority. First, it consistently prioritizes sending high-priority requests ahead of non-priority requests such as `st.rowsp.nop`. Second, within high-priority requests, it further prioritizes entries based on their `RowID`, sending the entry with the smallest `RowID` first. This ensures that entire rows of data for a specific token are transferred contiguously, allowing consumer-side expert computations to begin early on completed rows without waiting for the entire transfer to finish. This is particularly beneficial for long sequences, effectively hiding communication latency behind computation.

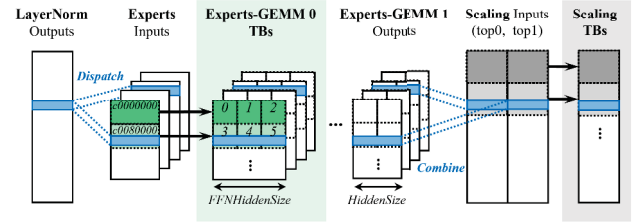
### C. Consumer: Data Availability Manager

While the RPM optimizes data transmission from the producer side, the Data Availability Manager (DAM) addresses the complementary challenge on the consumer side. The DAM replaces the costly software polling mechanism described in Section III-B with a hardware-managed, event-triggered signaling system, as illustrated in Fig. 11.

1) **Fine-Grained Dependency Tracking:** At the core of the DAM is a content-addressable memory (CAM)-based structure called the Dependency Table, which maps memory address ranges to sets of dependent Thread Blocks (TBs). This table is populated by the compiler through static analysis of the



(a) Data Availability Manager microarchitecture



(b) Dependency between data and subsequent kernel threadblocks

Fig. 11: Data Availability Manager microarchitecture and its role in managing dependencies between data and threadblocks.

consumer kernel. These dependencies stem from the consumer kernel's static tiling scheme and are not subject to the dynamic token order. Specifically, a TB that computes an output tile depends on the corresponding slice of the input activation matrix. Once that slice has received enough newly arrived token rows, the DAM can notify the TB scheduler to release the associated TB group. Fig. 11 illustrates this mapping between data address ranges and TB groups. For each model defined by its `HiddenSize` and `FFNHiddenSize`, and target hardware, the Dependency Table is generated once with auto-tuned tiling scheme and cached for reuse.

2) **Event-Triggered Thread Block Scheduling:** For each unique TB group recorded in the Dependency Table, a TB Status Counter is maintained. When a write acknowledgment (from a regular remote store `st.` or a `st.rowsp`) returns to the hub, the DAM performs a range lookup in the Dependency Table. For every matching entry, the corresponding TB Status Counter is incremented. Once a counter reaches a predefined threshold, indicating that all data required by the associated TB group is available in local memory, the DAM issues a Ready signal for that TB group to the GPU's thread block scheduler. These TBs then become immediately eligible for dispatch onto available Streaming Multiprocessors (SMs). By replacing software polling with hardware-managed readiness signaling, the DAM eliminates busy-waiting, freeing compute resources and memory bandwidth for productive computation.

3) **Runtime Adaptation to Dynamic Workloads:** The dynamic token routing in MoE models implies that the actual number of tokens processed by each expert is not known statically. To prevent resource under-utilization or

TABLE II: MoE model Configurations used in evaluation.

| Model Name     | Layer | Hidden Size | FFN Hidden Size | TopK / Experts |
|----------------|-------|-------------|-----------------|----------------|
| Mixtral 8x7B   | 32    | 4096        | 14336           | 2 / 8          |
| Qwen2-MoE-2.7B | 24    | 2048        | 1408            | 4 / 64         |
| Phi-3.5-MoE    | 32    | 4096        | 6400            | 2 / 16         |

oversubscription, the DAM incorporates a global completion detection mechanism. A Global Counter tracks the total number of write acknowledgments expected for a consumer kernel (e.g.,  $\text{HiddenSize} * \text{SequenceLength} * k$ ). When this counter reaches its target, an `AllReady` signal is generated. This signal enables the system to identify and deallocate TBs whose corresponding counters remain at zero. These TBs, spawned as a result of the compiler’s conservative allocation strategy, are never scheduled, thereby avoiding unnecessary consumption of SM resources. This runtime adaptation is essential for handling the variable workloads typical of MoE experts, ensuring that hardware resources are dedicated strictly to useful computation.

## V. EXPERIMENTAL METHODOLOGY

### A. Hardware Configuration

We model multi-GPU systems using an extended version of Accel-Sim [29], following the methodology of prior work [28], [51]. Our baseline configuration consists of 8 GPUs connected through 4 NVSwitch chips, emulating the NVIDIA DGX-H800 architecture features and topology.

The multi-GPU interconnect is simulated using a modified BookSim2 [25] that replicates NVLink’s design, featuring full-duplex links, 16B flits, and single-flit headers. The network supports full-to-full routing and switch-level forwarding. Each GPU is configured with 400 GB/s bandwidth, and the unidirectional latency between GPUs and switches is set to 250 ns, resulting in an approximate round-trip latency of 1  $\mu$ s. We calibrated both the bandwidth and latency of the simulated interconnect against real hardware measurements. Across message sizes ranging from 0.5 MB to 256 MB, the simulated All-to-All communication time exhibits an average error of 4.36% compared to physical system results.

### B. Workloads

We evaluate MoE-Hub using three representative MoE models, summarized in Table II. To manage simulation complexity, we focus on a single MoE layer; other end-to-end overheads are extrapolated from runtime breakdowns measured on real H800 systems. Expert GEMM operations are implemented using CUTLASS [43] kernels.

### C. Baseline

We compare MoE-Hub against five state-of-the-art MoE systems and an idealized upper bound. All baselines are implemented on top of Megatron-LM [59], a widely adopted transformer training framework that supports MoE models.

- 1) **Megatron-TE** [59] is a non-overlapping MoE implementation that integrates NVIDIA’s Transformer Engine [48] for optimized transformer kernels.

- 2) **FasterMoE** [16] improves MoE performance through optimized All-to-All and graph-level overlap between communication and expert computation.
- 3) **Tutel** [19] implements adaptive parallelism, pipelining, and a hierarchical 2D All-to-All algorithm to improve MoE scalability.
- 4) **Comet** [67] achieves fine-grained overlap between expert GEMM and All-to-All using tile-level dependency analysis, task rescheduling, and adaptive SM allocation.
- 5) **CCFuser** [61] fuses expert GEMM and All-to-All via inter-GPU shared memory access and improves GPU utilization through dynamic expert reassignment.
- 6) **Ideal MoE layer** represents the execution time with *ideal MoE layers* (Section II-C) consisting only of local routing, local expert computation, and local scaling.

## VI. RESULTS

### A. End-to-end Speedup

We evaluate MoE-Hub against five state-of-the-art MoE systems and an idealized lower bound across three representative models under varying GPU counts (NGPU) and sequence lengths. As all baselines share the same attention module implementation, performance differences arise solely from the MoE layer execution, including overhead from the address resolution phases (e.g. system synchronization).

Fig. 12 summarizes the end-to-end speedups. MoE-Hub achieves average speedups of 1.625 $\times$ , 1.984 $\times$ , 1.506 $\times$ , 1.209 $\times$ , and 1.278 $\times$  over Megatron-TE, FasterMoE, Tutel, Comet, and CCFuser, respectively. Since the attention is identical across baselines, these gains on the pure MoE layer are further amplified to 2.195 $\times$ , 3.082 $\times$ , 2.014 $\times$ , 1.400 $\times$ , and 1.542 $\times$ .

Megatron-TE and FasterMoE exhibit the lowest performance, as the former lacks computation-communication overlap while the latter relies on coarse-grained dataflow scheduling with limited model support. Tutel improves performance through graph-level overlap and adaptive parallelism. Comet and CCFuser, which employ fine-grained software optimizations, outperform the graph-level approaches. Nonetheless, MoE-Hub consistently surpasses all software-based systems, achieving 96.8% of the performance of the theoretical ideal MoE layer, which assumes no scheduling overhead or exposed inter-GPU communication.

Overall, the performance benefits of MoE-Hub primarily arise from two factors: (1) elimination of address resolution phases, including synchronization between devices and additional data shuffling, thereby reducing the intrinsic overhead of routing stages. (2) hardware-managed dataflow orchestration, removing software scheduling costs for fine-grained pipeline coordination using memory barriers, and for data availability tracking that would incur substantial polling overhead. While prior work has partially addressed the second aspect, MoE-Hub is the first to identify the root cause of the first issue and propose a thorough solution, enabling seamless and efficient fine-grained overlap.

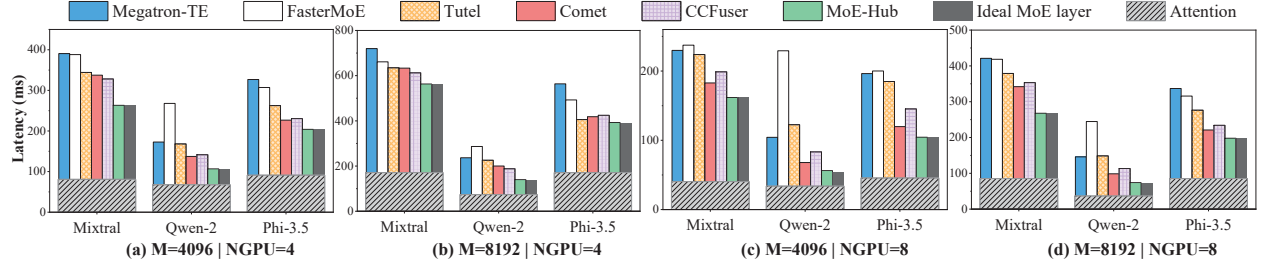


Fig. 12: End-to-end latency evaluation. Total token count is  $M = \text{SeqLength} \times \text{NGPU}$ , where NGPU is the number of devices.

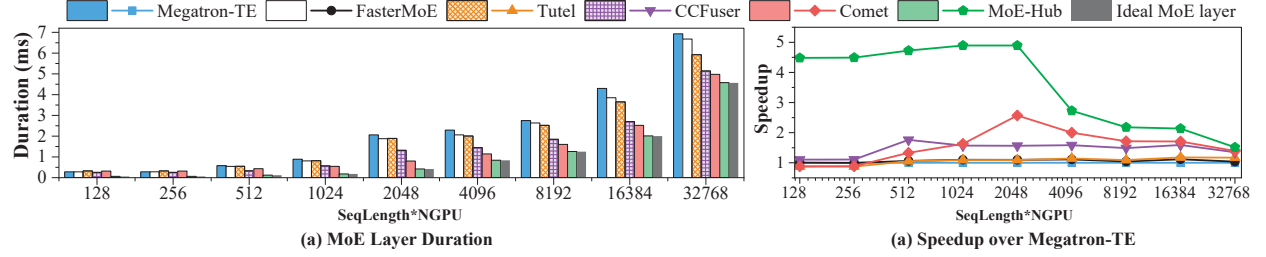


Fig. 13: Layer duration and speedup with varying total token count on 8 GPUs. Other parameters follow Mixtral 8 $\times$ 7B.

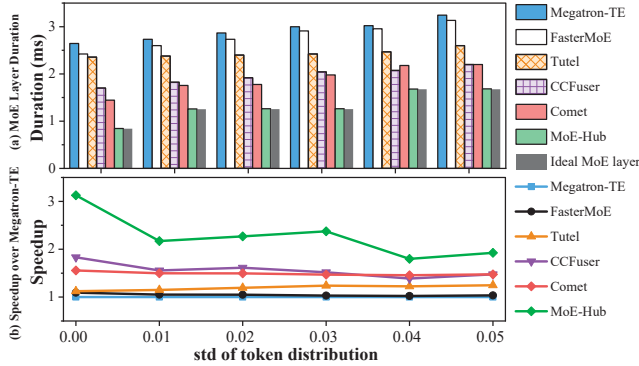


Fig. 14: Layer duration and speedup with a total token count of 8192 on 8 GPUs. Other parameters follow Mixtral 8 $\times$ 7B.

### B. Single MoE layer analysis

We further analyze a single MoE layer to evaluate the adaptability of MoE-Hub across different scenarios.

*a) Varying Sequence Length:* Fig. 13 reports MoE-layer execution time on 8 GPUs as the total number of input tokens increases from 128 to 32,768, along with the speedup over the non-overlapping Megatron-TE baseline. MoE-Hub delivers an average speedup of 4.70 $\times$  for small batches (128–2,048 tokens) and 2.14 $\times$  for larger batches (4,096–32,768 tokens), with an overall average of 3.56 $\times$  across the full range. At small token counts where communication volume is low, the layer is dominated by exposed per-step control-plane overhead rather than bulk data movement. Consequently, software approaches that primarily target overlapping communication provide limited benefit. MoE-Hub reduces these overheads by removing software-mediated scheduling and dataflow orchestration, resulting in substantially lower latency and higher speedup relative to other baselines. For large-batch

scenarios, expert GEMM occupies an increasing fraction of the layer execution time as token count grows, diminishing the relative benefit of effective communication overlap, and thus the speedups of fine-grained overlap works start to decline. Even in this regime, MoE-Hub continues to outperform prior approaches by eliminating data shuffling and other routing-induced operations whose cost scales with sequence length.

*b) Varying Token Distribution:* A key challenge for MoE-Hub is handling scheduling inefficiencies caused by dynamic producer-consumer relationships and irregular data flows in MoE models. We evaluate its performance under varying degrees of irregularity, quantified by the standard deviation (std) of token counts across experts. In typical training workloads, the average std is around 0.032 [67]. Fig. 14 presents results for std values from 0 to 0.05. Although latency increases with higher imbalance for all implementations, MoE-Hub maintains a consistent performance advantage across all std values, demonstrating robustness under different load imbalance conditions.

### C. Ablation study

We analyze the effectiveness of the proposed optimization techniques by comparing four MoE-Hub variants against Comet as the baseline:

- **MH-Base:** Base MoE-Hub with the destination-agnostic paradigm, where dataflow orchestration is software-managed.
- **MH-PKT:** MH-Base with hardware packet management.
- **MH-DEP:** MH-Base with hardware signaling for dependency-driven thread block dispatch.
- **MH:** Full MoE-Hub design incorporating both hardware packet management and hardware signaling.

We compare these four variants against Comet across three models and sequence lengths ranging from 4,096 to 16,384.

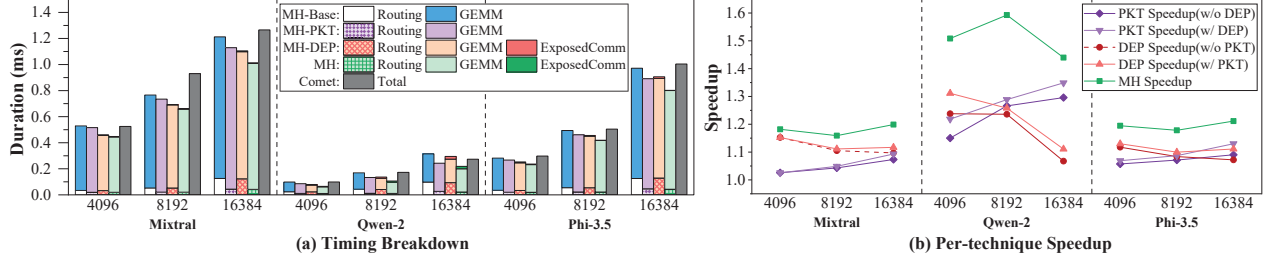


Fig. 15: Comparison of four MoE-Hub variants with Comet from routing to expert GEMM1, and speedups achieved by Runtime Packet Management, Hardware Signaling, and the full design.

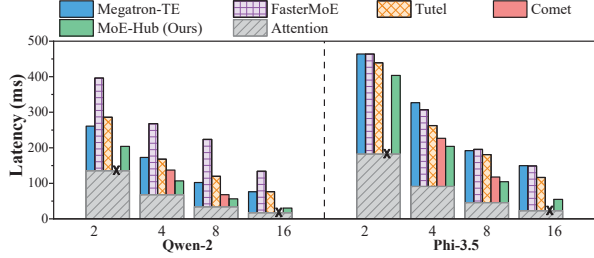


Fig. 16: End-to-end latency evaluation on Qwen-2 and Phi-3.5 across 2–16 GPUs, with a total token count of 8192.

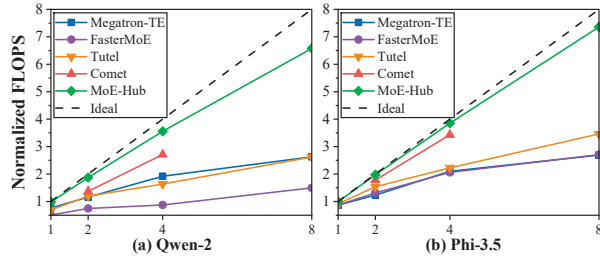


Fig. 17: Normalized FLOPS for 1x, 2x, 4x, and 8x GPU scaling, with the dashed line indicating ideal FLOPS scaling.

Our analysis focuses on the interval from routing to the end of the first expert GEMM (GEMM1), which is the most sensitive window for dataflow techniques. Because the routing phase is short and leaves limited compute slack, any exposed communication or orchestration overhead directly delays GEMM1.

As shown in Fig. 15, hardware packet management (**PKT**) consistently improves performance, achieving an average speedup of 1.13x. Its benefit grows with sequence length, regardless of whether **DEP** is applied. As sequence length increases, communication volume rises. Without packet management, transfers become burstier and less ordered, amplifying interconnect queuing and prolonging the routing to GEMM1 window. This effect is especially pronounced for models with smaller experts (e.g., Qwen-2), where locally available tokens alone cannot saturate the GPU, and delayed remote token arrivals directly translate into exposed stalls. **PKT** reduces All-to-All latency, smooths arrivals and pushes execution closer to seamless overlap.

Hardware signaling (**DEP**) achieves an average speedup of 1.14x as shown in Fig. 15. In contrast to **PKT**, **DEP** yields

the largest relative gain at smaller sequence lengths, gradually tapering as sequence length increases, regardless of whether **PKT** is applied. In software approaches, the consumer kernel relies on repeated atomic polling to detect data readiness, which adds control overhead and can expose communication time in the critical window. **DEP** replaces polling with hardware-managed dependency tracking, reducing this per-step control cost. As token count increases, larger GEMM workload amortizes the control overhead, so the benefit of signaling becomes less pronounced, while remaining consistently positive across models.

#### D. Scalability

Fig. 16 presents the scalability of MoE-Hub across 2 to 16 GPUs. In all configurations, MoE-Hub surpasses the baseline implementations. Owing to the high simulation cost, we did not extend the evaluation to larger GPU clusters. Instead, we normalized computational performance to highlight the effective throughput scaling trends as the number of GPUs increases. The results in Fig. 17 demonstrate that MoE-Hub maintains strong scalability across model configurations of both Qwen and Phi, achieving performance closer to the theoretical scaling curve than all baselines. This is attributed to MoE-Hub’s elimination of operations that impede scalability, such as CPU interference or inter-device synchronization, thereby enabling near-linear performance growth as system size increases.

#### E. Hardware Overhead

We evaluate hardware overhead using TSMC’s 7 nm technology. As shown in Fig. 7, MoE-Hub’s major hardware modifications are centralized extensions to the GPU’s existing hub unit. Within the hub, the Row Allocation Table accounts for the majority of area and is implemented as a 16-bank SRAM with dual ports. Outside the hub, building upon the existing store instruction datapath, integration into GPU pipeline adds only lightweight logic for decoding and routing the new instruction, along with the logic required to trigger the thread block dispatcher. In total, all hardware support occupies just 0.49 mm<sup>2</sup>, accounting for less than 0.06% of an H800 GPU’s die area, demonstrating that the proposed design is both feasible and area-efficient.

## VII. DISCUSSIONS

### A. Compatibility with MoE optimizations

MoE-Hub targets the communication and data-movement orchestration overhead induced by MoE routing dynamics. This communication-centric focus is complementary to computation-oriented MoE optimizations [5], [14], [39], [52], [63], [65], [66], including parallelism strategies and expert imbalance mitigation.

1) *EP+TP Hybrid Parallelism*: Large-scale training and inference often combine tensor parallelism (TP) with expert parallelism. MoE-Hub can directly support such scenarios, where dynamic routing still leads to runtime-determined address layouts. Specifically, an expert is sharded across a TP group in the TP+EP configuration. For each token and selected expert, the input data should be delivered to all GPUs in the TP group. MoE-Hub treats each TP shard as a distinct consumer endpoint, issuing one `rowspMalloc` per shard (i.e. one `MallocID` per GPU in the TP group) and emitting `st.rowsp` requests accordingly for each `MallocID`. The same hub-managed dataflow orchestration is retained.

2) *Cooperation with Load Balancing Mechanisms*: Modern MoE systems mitigate load imbalance via expert replication and dynamic expert placement [5], [39], [63], [65], [66]. Dynamics of routing and scheduling complexity of data movement persist under these optimizations. MoE-Hub's core mechanism can likewise accelerate these scenarios. The key difference is that the static one-to-one expert-to-device mapping is turned into a dynamic one-to-many mapping. This changes how a logical destination resolves to a consumer endpoint. MoE-Hub can be extended to manage these complex relationships in hub-local tables (e.g., by extending APT) and to support selection policies between endpoints (e.g., round-robin or load-aware policy). Furthermore, MoE-Hub expands the design space for MoE optimizations, enabling mechanisms that were previously impractical (e.g., expert migration is limited to a coarse granularity due to heavy software overhead [39], [65], [66]). We leave these extensions to future work.

### B. Beyond MoE

MoE-Hub's core destination-agnostic communication paradigm and hub-centric design are applicable to other workloads with runtime-dependent sparse or dynamic communication patterns. An emerging example is distributed KV-cache exchanges [24], [35], [64], where sparse blocks are exchanged across devices based on runtime results. While these workloads share the same motivating characteristics, supporting them requires detailed refinements and further explorations. As models become increasingly sparse and dynamic, hub-managed, destination-agnostic abstractions can provide a general substrate for sustaining efficiency beyond dense, static workloads.

## VIII. RELATED WORK

1) *Computation-Communication Overlapping*: Overlapping communication with computation is a widely studied

method to address communication challenges in distributed AI systems, explored in various parallelism schemes [6], [17], [21], [51], [62]. CoCoNet [21] first introduces kernel fusion to overlap AllReduce with GEMM operations. Centauri [6] employs multi-dimensional communication partitioning and hierarchical dataflow-graph scheduling to promote overlap in hybrid parallelism. T3 [51] leverages hardware primitives for overlap in tensor parallelism. However, these approaches are primarily designed for regular large-model workloads and do not address the challenges introduced by the inherent dynamism of MoE models.

2) *MoE-Specific Optimizations*: Early MoE systems enforced fixed token capacities per expert to simplify compilation and execution [31], [53], though this often came at the cost of model accuracy [12], [13]. Later systems introduced CPU-assisted dynamic shape support, which enabled a range of optimizations to study dynamic communication bottlenecks in MoE, including improving the implementation of All-to-All operations [19], [58], [68], [69] and exploring compute-communication overlap strategies at multiple granularities, ranging from graph-level overlap (e.g., FasterMoE [16], Tutel [19]) to finer-grained overlap (e.g., Comet [67], CC-Fuser [61]). Recent work has identified the significant overhead of CPU coordination and attempts to build fully GPU-driven computation-communication pipelines, such as via GPU-side address synchronization (e.g., Primus-Turbo [3]) or kernel fusion (e.g., FlashDMoE [2]). Nevertheless, the overlap mechanisms proposed by these works often impose trade-offs between performance and programmability. In contrast, MoE-Hub removes address-resolution-induced synchronization from the critical path and uses hardware-managed dataflow orchestration to achieve efficient fine-grained overlap without burdening the software stack.

## IX. CONCLUSION

In this paper, we propose MoE-Hub, a hardware-software co-design that addresses inefficiencies in distributed MoE models caused by inter-GPU communication bottlenecks. By decoupling data transmission from address management and hardware-accelerating the communication data flow, MoE-Hub enables seamless and high-performance overlap of computation and communication. MoE-Hub achieves  $1.40\times$ – $3.08\times$  speedup per MoE layer and  $1.21\times$ – $1.98\times$  end-to-end speedup over state-of-the-art works, offering a more efficient solution for large-scale MoE models.

## X. ACKNOWLEDGMENT

We sincerely thank the anonymous ISCA'26 reviewers for their valuable suggestions that improved the paper. This work is supported by National Natural Science Foundation of China (NSFC) grant (62502305), Natural Science Foundation of Shanghai (NSFS) grant 25ZR1402275, Shanghai QiYuan Innovation Foundation QY2025-QN-SJTU-011, and Shanghai Qi Zhi Institute Innovation Program SQZ202316.

## REFERENCES

- [1] M. Abdin, J. Aneja, H. Awadalla, A. Awadallah, A. A. Awan, N. Bach, A. Bahree, A. Bakhtiari, J. Bao, H. Behl, A. Benhaim, M. Bilenko, J. Bjorck, S. Bubeck, M. Cai, Q. Cai, V. Chaudhary, D. Chen, D. Chen, W. Chen, Y.-C. Chen, Y.-L. Chen, H. Cheng, P. Chopra, X. Dai, M. Dixon, R. Eldan, V. Frago, J. Gao, M. Gao, M. Gao, A. Garg, A. D. Giorno, A. Goswami, S. Gunasekar, E. Haider, J. Hao, R. J. Hewett, W. Hu, J. Huynh, D. Iter, S. A. Jacobs, M. Javaheripi, X. Jin, N. Karampatziakis, P. Kauffmann, M. Khademi, D. Kim, Y. J. Kim, L. Kurilenko, J. R. Lee, Y. T. Lee, Y. Li, Y. Li, C. Liang, L. Liden, X. Lin, Z. Lin, C. Liu, L. Liu, M. Liu, W. Liu, X. Liu, C. Luo, P. Madan, A. Mahmoudzadeh, D. Majercak, M. Mazzola, C. C. T. Mendes, A. Mitra, H. Modi, A. Nguyen, B. Norick, B. Patra, D. Perez-Becker, T. Portet, R. Pryzant, H. Qin, M. Radmilac, L. Ren, G. de Rosa, C. Rosset, S. Roy, O. Ruwase, O. Saarikivi, A. Saied, A. Salim, M. Santacrose, S. Shah, N. Shang, H. Sharma, Y. Shen, S. Shukla, X. Song, M. Tanaka, A. Tupini, P. Vaddamanu, C. Wang, G. Wang, L. Wang, S. Wang, X. Wang, Y. Wang, R. Ward, W. Wen, P. Witte, H. Wu, X. Wu, M. Wyatt, B. Xiao, C. Xu, J. Xu, W. Xu, J. Xue, S. Yadav, F. Yang, J. Yang, Y. Yang, Z. Yang, D. Yu, L. Yuan, C. Zhang, C. Zhang, J. Zhang, L. L. Zhang, Y. Zhang, Y. Zhang, Y. Zhang, and X. Zhou, "Phi-3 technical report: A highly capable language model locally on your phone," 2024. [Online]. Available: <https://arxiv.org/abs/2404.14219>
- [2] O. J. Aimuyo, B. Oh, and R. Singh, "Flashdmoe: Fast distributed moe in a single kernel," *arXiv preprint arXiv:2506.04667*, 2025.
- [3] AMD, "Moe training best practices on amd gpus." <https://rocm.blogs.amd.com/software-tools-optimization/primus-moe-package/README.html>, 2025.
- [4] J. Bai, S. Bai, S. Yang, S. Wang, S. Tan, P. Wang, J. Lin, C. Zhou, and J. Zhou, "Qwen-vl: A frontier large vision-language model with versatile abilities," *arXiv preprint arXiv:2308.12966*, vol. 1, no. 2, p. 3, 2023.
- [5] O. Balmau, A.-M. Kermarrec, R. Pires, A. L. E. Santo, M. de Vos, and M. Vujasinovic, "Accelerating moe model inference with expert sharding," in *Proceedings of the 5th Workshop on Machine Learning and Systems*, ser. EuroMLSys '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 192–199. [Online]. Available: <https://doi.org/10.1145/3721146.3721940>
- [6] C. Chen, X. Li, Q. Zhu, J. Duan, P. Sun, X. Zhang, and C. Yang, "Centauri: Enabling efficient scheduling for communication-computation overlap in large model training via communication partitioning," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024, pp. 178–191.
- [7] Y. Chen, B. Brock, S. Porumbescu, A. Buluç, K. Yelick, and J. D. Owens, "Scalable irregular parallelism with gpus: getting cpus out of the way," in *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, ser. SC '22. IEEE Press, 2022.
- [8] X. Cheng, Z. Zhang, Y. Zhou, J. Ji, J. Jiang, Z. Zhao, Z. Xiao, Z. Ye, Y. Huang, R. Lai, H. Jin, B. Hou, M. Wu, Y. Dong, A. Yip, Z. Ye, S. Wang, W. Yang, X. Miao, T. Chen, and Z. Jia, "Mirage persistent kernel: A compiler and runtime for mega-kernelizing tensor programs," 2025. [Online]. Available: <https://arxiv.org/abs/2512.22219>
- [9] D. Dai, C. Deng, C. Zhao, R. X. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu, Z. Xie, Y. K. Li, P. Huang, F. Luo, C. Ruan, Z. Sui, and W. Liang, "Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models," 2024. [Online]. Available: <https://arxiv.org/abs/2401.06066>
- [10] DeepSeek-AI, A. Liu, B. Feng, B. Wang, B. Wang, B. Liu, C. Zhao, C. Deng, C. Ruan, D. Dai, D. Guo, D. Yang, D. Chen, D. Ji, E. Li, F. Lin, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Xu, H. Yang, H. Zhang, H. Ding, H. Xin, H. Gao, H. Li, H. Qu, J. L. Cai, J. Liang, J. Guo, J. Ni, J. Li, J. Chen, J. Yuan, J. Qiu, J. Song, K. Dong, K. Gao, K. Guan, L. Wang, L. Zhang, L. Xu, L. Xia, L. Zhao, L. Zhang, M. Li, M. Wang, M. Zhang, M. Zhang, M. Tang, M. Li, N. Tian, P. Huang, P. Wang, P. Zhang, Q. Zhu, Q. Chen, Q. Du, R. J. Chen, R. L. Jin, R. Ge, R. Pan, R. Xu, R. Chen, S. S. Li, S. Lu, S. Zhou, S. Chen, S. Wu, S. Ye, S. Ma, S. Wang, S. Zhou, S. Yu, S. Zhou, S. Zheng, T. Wang, T. Pei, T. Yuan, T. Sun, W. L. Xiao, W. Zeng, W. An, W. Liu, W. Liang, W. Gao, W. Zhang, X. Q. Li, X. Jin, X. Wang, X. Bi, X. Liu, X. Wang, X. Shen, X. Chen, X. Chen, X. Nie, X. Sun, X. Wang, X. Liu, X. Xie, X. Yu, X. Song, X. Zhou, X. Yang, X. Lu, X. Su, Y. Wu, Y. K. Li, Y. X. Wei, Y. X. Zhu, Y. Xu, Y. Huang, Y. Li, Y. Zhao, Y. Sun, Y. Li, Y. Wang, Y. Zheng, Y. Zhang, Y. Xiong, Y. He, Y. Tang, Y. Piao, Y. Dong, Y. Tan, Y. Liu, Y. Wang, Y. Guo, Y. Zhu, Y. Wang, Y. Zou, Y. Zha, Y. Ma, Y. Yan, Y. You, Y. Liu, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Huang, Z. Zhang, Z. Xie, Z. Hao, Z. Shao, Z. Wen, Z. Xu, Z. Zhang, Z. Li, Z. Wang, Z. Gu, Z. Li, and Z. Xie, "Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model," 2024. [Online]. Available: <https://arxiv.org/abs/2405.04434>
- [11] DeepSeek-AI, A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, D. Dai, D. Guo, D. Yang, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Bao, H. Xu, H. Wang, H. Zhang, H. Ding, H. Xin, H. Gao, H. Li, H. Qu, J. L. Cai, J. Liang, J. Guo, J. Ni, J. Li, J. Wang, J. Chen, J. Chen, J. Yuan, J. Qiu, J. Li, J. Song, K. Dong, K. Hu, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Xu, L. Xia, L. Zhao, L. Wang, L. Zhang, M. Li, M. Wang, M. Zhang, M. Zhang, M. Tang, M. Li, N. Tian, P. Huang, P. Wang, P. Zhang, Q. Wang, Q. Zhu, Q. Chen, Q. Du, R. J. Chen, R. L. Jin, R. Ge, R. Zhang, R. Pan, R. Wang, R. Xu, R. Zhang, R. Chen, S. S. Li, S. Lu, S. Zhou, S. Chen, S. Wu, S. Ye, S. Ye, S. Ma, S. Wang, S. Zhou, S. Yu, S. Zhou, S. Pan, T. Wang, T. Yun, T. Pei, T. Sun, W. L. Xiao, W. Zeng, W. Zhao, W. An, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, X. Q. Li, X. Jin, X. Wang, X. Bi, X. Liu, X. Wang, X. Shen, X. Chen, X. Zhang, X. Chen, X. Nie, X. Sun, X. Wang, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yu, X. Song, X. Shan, X. Zhou, X. Yang, X. Li, X. Su, X. Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Y. Zhang, Y. Xu, Y. Xu, Y. Huang, Y. Li, Y. Zhao, Y. Sun, Y. Li, Y. Wang, Y. Yu, Y. Zheng, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Tang, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Wu, Y. Ou, Y. Zhu, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Zha, Y. Xiong, Y. Ma, Y. Yan, Y. Luo, Y. You, Y. Liu, Y. Zhou, Z. F. Wu, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Huang, Z. Zhang, Z. Xie, Z. Zhang, Z. Hao, Z. Gou, Z. Ma, Z. Yan, Z. Shao, Z. Xu, Z. Wu, Z. Zhang, Z. Li, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Gao, and Z. Pan, "Deepseek-v3 technical report," 2025. [Online]. Available: <https://arxiv.org/abs/2412.19437>
- [12] N. Du, Y. Huang, A. M. Dai, S. Tong, D. Lepikhin, Y. Xu, M. Krikun, Y. Zhou, A. W. Yu, O. Firat, B. Zoph, L. Fedus, M. P. Bosma, Z. Zhou, T. Wang, E. Wang, K. Webster, M. Peltat, K. Robinson, K. Meier-Hellstern, T. Duke, L. Dixon, K. Zhang, Q. Le, Y. Wu, Z. Chen, and C. Cui, "GLaM: Efficient scaling of language models with mixture-of-experts," in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu, and S. Sabato, Eds., vol. 162. PMLR, 17–23 Jul 2022, pp. 5547–5569. [Online]. Available: <https://proceedings.mlr.press/v162/du22c.html>
- [13] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.
- [14] T. Gale, D. Narayanan, C. Young, and M. Zaharia, "Megablocks: Efficient sparse training with mixture-of-experts," *Proceedings of Machine Learning and Systems*, vol. 5, pp. 288–304, 2023.
- [15] C. Guo, R. Zhang, J. Xu, J. Leng, Z. Liu, Z. Huang, M. Guo, H. Wu, S. Zhao, J. Zhao, and K. Zhang, "Gmlake: Efficient and transparent gpu memory defragmentation for large-scale dnn training with virtual memory stitching," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 450–466. [Online]. Available: <https://doi.org/10.1145/3620665.3640423>
- [16] J. He, J. Zhai, T. Antunes, H. Wang, F. Luo, S. Shi, and Q. Li, "Faster-moe: modeling and optimizing training of large-scale dynamic pre-trained models," in *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, 2022, pp. 120–134.
- [17] K. Hong, X. Li, M. Liu, Q. Mao, T. Wu, Z. Huang, L. Chen, Z. Wang, Y. Zhang, Z. Zhu, G. Dai, and Y. Wang, "Efficient and adaptable overlapping for computation and communication via signaling and reordering," 2025. [Online]. Available: <https://arxiv.org/abs/2504.19519>
- [18] H. Huang, N. Ardalani, A. Sun, L. Ke, H.-H. S. Lee, S. Bhosale, C.-J. Wu, and B. Lee, "Toward efficient inference for mixture of experts," *Advances in Neural Information Processing Systems*, vol. 37, pp. 84033–84059, 2024.
- [19] C. Hwang, W. Cui, Y. Xiong, Z. Yang, Z. Liu, H. Hu, Z. Wang,

- R. Salas, J. Jose, P. Ram, H. Chau, P. Cheng, F. Yang, M. Yang, and Y. Xiong, "Tutel: Adaptive mixture-of-experts at scale," in *Proceedings of Machine Learning and Systems*, D. Song, M. Carbin, and T. Chen, Eds., vol. 5. Curran, 2023, pp. 269–287. [Online]. Available: [https://proceedings.mlsys.org/paper\\_files/paper/2023/file/5616d34cf8ff73942cfd5aa922842556-Paper-mlsys2023.pdf](https://proceedings.mlsys.org/paper_files/paper/2023/file/5616d34cf8ff73942cfd5aa922842556-Paper-mlsys2023.pdf)
- [20] R. Hwang, J. Wei, S. Cao, C. Hwang, X. Tang, T. Cao, and M. Yang, "Pre-gated moe: An algorithm-system co-design for fast and scalable mixture-of-expert inference," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 1018–1031.
- [21] A. Jangda, J. Huang, G. Liu, A. H. N. Sabet, S. Maleki, Y. Miao, M. Musuvathi, T. Mytkowicz, and O. Saarikivi, "Breaking the computation and communication abstraction barrier in distributed machine learning workloads," in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2022, pp. 402–416.
- [22] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chlapot, D. de las Casas, E. B. Hanna, F. Bressand, G. Lengyel, G. Bour, G. Lample, L. R. Lavaud, L. Saulnier, M.-A. Lachaux, P. Stock, S. Subramanian, S. Yang, S. Antoniak, T. L. Scao, T. Gervet, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed, "Mixtral of experts," 2024. [Online]. Available: <https://arxiv.org/abs/2401.04088>
- [23] C. Jiang, Y. Tian, X. Jia, S. Zheng, C. Wu, and Y. Wang, "Lancet: Accelerating mixture-of-experts training via whole graph computation-communication overlapping," *Proceedings of Machine Learning and Systems*, vol. 6, pp. 74–86, 2024.
- [24] H. Jiang, Y. Li, C. Zhang, Q. Wu, X. Luo, S. Ahn, Z. Han, A. H. Abdi, D. Li, C.-Y. Lin, Y. Yang, and L. Qiu, "Minference 1.0: accelerating pre-filling for long-context llms via dynamic sparse attention," in *Proceedings of the 38th International Conference on Neural Information Processing Systems*, ser. NIPS '24. Red Hook, NY, USA: Curran Associates Inc., 2024.
- [25] N. Jiang, D. U. Becker, G. Micheliogiannakis, J. Balfour, B. Towles, D. E. Shaw, J. Kim, and W. J. Dally, "A detailed and flexible cycle-accurate network-on-chip simulator," in *2013 IEEE international symposium on performance analysis of systems and software (ISPASS)*. IEEE, 2013, pp. 86–96.
- [26] C. Jin, Z. Jiang, Z. Bai, Z. Zhong, J. Liu, X. Li, N. Zheng, X. Wang, C. Xie, Q. Huang, W. Heng, Y. Ma, W. Bao, S. Zheng, Y. Peng, H. Lin, X. Liu, J. Jin, and X. Liu, "Megascala-moe: Large-scale communication-efficient training of mixture-of-experts models in production," 2025. [Online]. Available: <https://arxiv.org/abs/2505.11432>
- [27] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020.
- [28] M. Khairy, V. Nikiforov, D. Nellans, and T. G. Rogers, "Locality-centric data and threadblock management for massive gpus," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 1022–1036.
- [29] M. Khairy, Z. Shen, T. M. Aamodt, and T. G. Rogers, "Accel-sim: An extensible simulation framework for validated gpu modeling," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2020, pp. 473–486.
- [30] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, ser. SOSP '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 611–626. [Online]. Available: <https://doi.org/10.1145/3600006.3613165>
- [31] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen, "Gshard: Scaling giant models with conditional computation and automatic sharding," *arXiv preprint arXiv:2006.16668*, 2020.
- [32] J. Li, Y. Jiang, Y. Zhu, C. Wang, and H. Xu, "Accelerating distributed MoE training and inference with lina," in *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. Boston, MA: USENIX Association, Jul. 2023, pp. 945–959. [Online]. Available: <https://www.usenix.org/conference/atc23/presentation/li-jiamin>
- [33] B. Lin, C. Zhang, T. Peng, H. Zhao, W. Xiao, M. Sun, A. Liu, Z. Zhang, L. Li, X. Qiu, S. Li, Z. Ji, T. Xie, Y. Li, and W. Lin, "Infinite-llm: Efficient llm service for long context with distattention and distributed kvcache," 2024. [Online]. Available: <https://arxiv.org/abs/2401.02669>
- [34] J. Liu, J. H. Wang, and Y. Jiang, "Janus: A unified distributed training framework for sparse mixture-of-experts models," in *Proceedings of the ACM SIGCOMM 2023 Conference*, ser. ACM SIGCOMM '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 486–498. [Online]. Available: <https://doi.org/10.1145/3603269.3604869>
- [35] E. Lu, Z. Jiang, J. Liu, Y. Du, T. Jiang, C. Hong, S. Liu, W. He, E. Yuan, Y. Wang, Z. Huang, H. Yuan, S. Xu, X. Xu, G. Lai, Y. Chen, H. Zheng, J. Yan, J. Su, Y. Wu, N. Y. Zhang, Z. Yang, X. Zhou, M. Zhang, and J. Qiu, "Moba: Mixture of block attention for long-context llms," 2025. [Online]. Available: <https://arxiv.org/abs/2502.13189>
- [36] Meta AI, "The llama 4 herd: The beginning of a new era of natively multimodal ai innovation," <https://ai.meta.com/blog/llama-4-multimodal-intelligence/>, checked on, vol. 4, no. 7, p. 2025, 2025.
- [37] H. Muthukrishnan, D. Lustig, O. Villa, T. Wenisch, and D. Nellans, "Finepack: Transparently improving the efficiency of fine-grained transfers in multi-gpu systems," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 516–529.
- [38] H. Muthukrishnan, D. Nellans, D. Lustig, J. A. Fessler, and T. F. Wenisch, "Efficient multi-gpu shared memory via automatic optimization of fine-grained transfers," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 139–152.
- [39] X. Nie, X. Miao, Z. Wang, Z. Yang, J. Xue, L. Ma, G. Cao, and B. Cui, "Flexmoe: Scaling large-scale sparse pre-trained model training via dynamic device placement," *Proc. ACM Manag. Data*, vol. 1, no. 1, May 2023. [Online]. Available: <https://doi.org/10.1145/3588964>
- [40] NVIDIA, "Peer-to-peer & unified virtual addressing," [https://developer.download.nvidia.com/CUDA/training/cuda\\_webinars\\_GPU\\_Direct\\_uva.pdf](https://developer.download.nvidia.com/CUDA/training/cuda_webinars_GPU_Direct_uva.pdf), 2011.
- [41] —, "Doubling all2all performance with nvidia collective communication library 2.12," <https://developer.nvidia.com/blog/doubling-all2all-performance-with-nvidia-collective-communication-library-2-12/>, 2022.
- [42] —, "Applying mixture of experts in llm architectures," <https://developer.nvidia.com/blog/applying-mixture-of-experts-in-llm-architectures/>, 2024.
- [43] —, "Cuda templates for linear algebra subroutines," <https://github.com/NVIDIA/cutlass>, 2024.
- [44] —, "Introduction to nvidia dgx h100/h200 systems," <https://docs.nvidia.com/dgx/dgxh100-user-guide/introduction-to-dgxh100.html>, 2024.
- [45] —, "New llm: Snowflake arctic model for sql and code generation," <https://developer.nvidia.com/blog/new-llm-snowflake-arctic-model-for-sql-and-code-generation/>, 2024.
- [46] —, "Mixture of experts powers the most intelligent frontier ai models, runs 10x faster to deliver 1/10 the token cost on nvidia blackwell nv172," <https://blogs.nvidia.com/blog/mixture-of-experts-frontier-models/>, 2025.
- [47] —, "Nvidia nvshmem library (nvshmem) documentation," <https://docs.nvidia.com/nvshmem/api/index.html>, 2025.
- [48] —, "Transformer engine," <https://github.com/NVIDIA/TransformerEngine>, 2025.
- [49] OpenAI, "Gpt-oss," <https://github.com/openai/gpt-oss>, 2025.
- [50] —, "Introducing gpt-5," <https://openai.com/index/introducing-gpt-5>, 2025.
- [51] S. Pati, S. Aga, M. Islam, N. Jayasena, and M. D. Sinclair, "T3: Transparent tracking & triggering for fine-grained overlap of compute & collectives," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2024, pp. 1146–1164.
- [52] S. Rajbhandari, C. Li, Z. Yao, M. Zhang, R. Y. Aminabadi, A. A. Awan, J. Rasley, and Y. He, "DeepSpeed-moe: Advancing mixture-of-experts inference and training to power next-generation ai scale," in *International conference on machine learning*. PMLR, 2022, pp. 18 332–18 346.
- [53] J. Rasley, S. Rajbhandari, O. Ruwase, and Y. He, "DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters," in *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, 2020, pp. 3505–3506.
- [54] M. Research, "Introducing dbrx: A new state-of-the-art open llm," <https://www.databricks.com/blog/introducing-dbrx-new-state-art-open-llm>, 2024.
- [55] C. Riquelme, J. Puigcerver, B. Mustafa, M. Neumann, R. Jenatton, A. Susano Pinto, D. Keysers, and N. Houlsby, "Scaling vision with

- sparse mixture of experts,” *Advances in Neural Information Processing Systems*, vol. 34, pp. 8583–8595, 2021.
- [56] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, “The sparsely-gated mixture-of-experts layer,” *Outrageously large neural networks*, vol. 2, 2017.
- [57] S. Shi, X. Pan, X. Chu, and B. Li, “Pipemoe: Accelerating mixture-of-experts through adaptive pipelining,” in *IEEE INFOCOM 2023-IEEE Conference on Computer Communications*. IEEE, 2023, pp. 1–10.
- [58] S. Shi, X. Pan, Q. Wang, C. Liu, X. Ren, Z. Hu, Y. Yang, B. Li, and X. Chu, “Schemoe: An extensible mixture-of-experts distributed training system with tasks scheduling,” in *Proceedings of the Nineteenth European Conference on Computer Systems*, 2024, pp. 236–249.
- [59] M. Shoneybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, “Megatron-lm: Training multi-billion parameter language models using model parallelism,” *arXiv preprint arXiv:1909.08053*, 2019.
- [60] Y. Tang, Y. Yin, Y. Wang, H. Zhou, Y. Pan, W. Guo, Z. Zhang, M. Rang, F. Liu, N. Zhang, B. Li, Y. Dong, X. Meng, Y. Wang, D. Li, Y. Li, D. Tu, C. Chen, Y. Yan, F. Yu, R. Tang, Y. Wang, B. Huang, B. Wang, B. Liu, C. Zhang, D. Kuang, F. Liu, G. Huang, J. Wei, J. Qin, J. Ran, J. Li, J. Zhao, L. Dai, L. Li, L. Deng, P. Qin, P. Zeng, Q. Gu, S. Tang, S. Cheng, T. Gao, T. Yu, T. Li, T. Bi, W. He, W. Mao, W. Huang, W. Liu, X. Li, X. Yu, X. Wu, X. He, Y. Du, Y. Xu, Y. Tian, Y. Wu, Y. Huang, Y. Tian, Y. Zhu, Y. Li, Y. Wang, Y. Gai, Y. Li, Y. Luo, Y. Ni, Y. Sun, Z. Chen, Z. Liu, Z. Liu, Z. Tu, Z. Ding, and Z. Zhan, “Pangu ultra moe: How to train your big moe on ascend npus,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.04519>
- [61] H. Wang, Y. Xia, D. Yang, X. Zhou, and D. Cheng, “Harnessing inter-gpu shared memory for seamless moe communication-computation fusion,” in *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, 2025, pp. 170–182.
- [62] S. Wang, J. Wei, A. Sabne, A. Davis, B. Ilbeyi, B. Hechtman, D. Chen, K. S. Murthy, M. Maggioni, Q. Zhang, S. Kumar, T. Guo, Y. Xu, and Z. Zhou, “Overlap communication with dependent computation via decomposition in large deep learning models,” in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ser. ASPLOS 2023. New York, NY, USA: Association for Computing Machinery, 2022, p. 93–106. [Online]. Available: <https://doi.org/10.1145/3567955.3567959>
- [63] W. Wang, Z. Lai, S. Li, W. Liu, K. Ge, Y. Liu, A. Shen, and D. Li, “Prophet: Fine-grained load balancing for parallel training of large-scale moe models,” in *2023 IEEE International Conference on Cluster Computing (CLUSTER)*, 2023, pp. 82–94.
- [64] R. Xu, G. Xiao, H. Huang, J. Guo, and S. Han, “Xattention: Block sparse attention with antidiagonal scoring,” 2025. [Online]. Available: <https://arxiv.org/abs/2503.16428>
- [65] D. Yu, L. Shen, H. Hao, W. Gong, H. Wu, J. Bian, L. Dai, and H. Xiong, “Moesys: A distributed and efficient mixture-of-experts training and inference system for internet services,” *IEEE Transactions on Services Computing*, vol. 17, no. 5, pp. 2626–2639, 2024.
- [66] M. Zhai, J. He, Z. Ma, Z. Zong, R. Zhang, and J. Zhai, “SmartMoE: Efficiently training Sparsely-Activated models through combining offline and online parallelization,” in *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. Boston, MA: USENIX Association, Jul. 2023, pp. 961–975. [Online]. Available: <https://www.usenix.org/conference/atc23/presentation/zhai>
- [67] S. Zhang, N. Zheng, H. Lin, Z. Jiang, W. Bao, C. Jiang, Q. Hou, W. Cui, S. Zheng, L.-W. Chang, Q. Chen, and X. Liu, “Comet: Fine-grained computation-communication overlapping for mixture-of-experts,” in *Proceedings of Machine Learning and Systems*, M. Zaharia, G. Joshi, and Y. Lin, Eds., vol. 7. MLSys, 2025. [Online]. Available: [https://proceedings.mlsys.org/paper\\_files/paper/2025/file/e27ea0cd50b798ff8942caf9203f0992-Paper-Conference.pdf](https://proceedings.mlsys.org/paper_files/paper/2025/file/e27ea0cd50b798ff8942caf9203f0992-Paper-Conference.pdf)
- [68] Z. Zhang, Y. Xia, H. Wang, D. Yang, C. Hu, X. Zhou, and D. Cheng, “Mpmoe: Memory efficient moe for pre-trained models with adaptive pipeline parallelism,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 35, no. 6, pp. 998–1011, 2024.
- [69] C. Zhao, S. Zhou, L. Zhang, C. Deng, Z. Xu, Y. Liu, K. Yu, J. Li, and L. Zhao, “Deepep: an efficient expert-parallel communication library,” <https://github.com/deepseek-ai/DeepEP>, 2025.